

혼자서 팀처럼 (미리보기)

Claude Code 멀티 에이전트 오케스트레이션 실전

codex · GLM · Gemini 워커 풀을 Claude가 지휘하게 만들어, 한도는 아끼고 구
현은 병렬로

v1.0 · 2026

이 책에 대하여

이 책은 Claude Code 세션 하나를 지휘관으로 두고 `codex` · `GLM` · `Gemini CLI` 워커 풀을 병렬로 운용하는 실제 운영 체계를 다룹니다. 모든 규칙은 2026년 7~8월 실운영에서 겪은 사고와 그 복구 과정에서 나왔습니다.

이 책은 특정 도구의 성과나 수익을 보장하지 않습니다. 모델명·CLI 플래그는 2026-08 기준이며, 도구가 바뀌면 개정판을 같은 구매 링크에서 무료로 받을 수 있습니다. 계정 생성, 결제, 비밀번호 처리 같은 명의 행위는 반드시 사람이 직접 수행하십시오.

구매자 파일: 본문 PDF, `templates.zip`(`CLAUDE.md` 오케스트레이션 섹션, `worker_watch.sh`, `GLM` 래퍼, 리뷰 게이트 프롬프트, 작업 명세·진행 보고 템플릿, 안전 금지 목록). 환불은 구매 후 14일 이내 사유 불문.

목차

0. 들어가며: 왜 "지휘관 + 워커 풀"인가

- 지휘관/워커 분리의 3가지 이득 · 첫날부터 적용할 최소 운영 루프 · 이 책의 구조와 읽는 법 · 예고: 피로 쓴 3건의 실제 사고 사례

1. 역할 분리 설계: Claude는 지휘관, 나머지는 손

- 1. 한 세션 만능주의를 버리는 이유 · 2. 왜 Claude 안의 서브에이전트가 아니라 외부 CLI 워커인가 · 3. Claude가 직접 할 일과 위임할 일 · 4. 워커 풀은 우선순위 체인이 아닙니다 · 5. 지휘관의 5줄 규칙 · 6. 완료와 재배정은 별도 운영 규칙입니다 · 7. 안전한 지휘 범위

2. 30분 세팅: 워커 5기 설치·인증·래퍼

- 1. 세팅 전에 정할 경계 · 2. 실제 설치와 구독 경계 · 3. codex CLI: Sol·Terra·Luna를 분리해 호출하기 · 4. Claude Code 헤드리스 워커와 GLM 래퍼 · 5. agy와 Gemini 계열 보조 워커 · 6. 셸 시작 파일에 안전하게 연결하기 · 7. 첫 병렬 실행을 위한 운영 규칙

0. 들어가며: 왜 "지휘관 + 워커 풀"인가

여러분은 지금 Claude Code를 켜고 "이 기능을 만들어줘"라고 지시합니다. 처음에는 마법처럼 코드를 작성합니다. 하지만 파일이 10개, 20개 늘어나고 디버깅 루프가 길어지면 어떤 일이 발생합니까? 곧 "주간 사용량 한도(Weekly Limit)에 도달했습니다"라는 메시지를 만나게 됩니다. 혹은 에이전트가 20분째 멈춰 서서 혼자 생각만 하고 있습니다. 컨텍스트 윈도우는 과거의 실패한 시도로 오염되어, 이제는 맞는 코드도 틀리게 고치기 시작합니다.

단일 세션이 모든 것을 처리하는 방식은 장난감 프로젝트에서는 통하지만, 실전에서는 곧 한계에 부딪힙니다. 단일 모델에 의존하는 방식의 근본적인 문제는 세 가지입니다. 첫째, 한도(Limit) 문제입니다. 고성능 모델을 코드를 훑고 단순 타이핑을 하는 데 낭비하면 정작 깊은 추론이 필요한 순간에 한도가 바닥납니다. 둘째, 대기 시간(Latency)입니다. 에이전트가 파일을 읽고 고치는 동안 사람은 화면만 쳐다봐야 합니다. 셋째, 컨텍스트 오염입니다. 실패한 시도와 무관한 파일의 내용이 누적되면서 모델의 판단력이 급격히 저하됩니다.

이 책은 이러한 한계를 돌파하기 위한 **오늘 밤 그대로 복붙해서 돌리는 운영 매뉴얼**입니다. 핵심은 간단합니다. Claude를 직접 코딩하는 노동자가 아니라, 기획하고 분배하고 검증하는 **지휘관(Orchestrator)**으로 격상시키는 것입니다. 그리고 실제 코드를 작성하고 탐색하는 일은 codex, GLM, agy 등 여러 워커(Worker) 모델에게 병렬로 위임합니다.

지휘관/워커 분리의 3가지 이득

1. 한도 분산과 병렬 처리량. Claude의 주간 한도는 기획, 검토, 지휘처럼 판단 가치가 큰 일에 집중합니다. 실제 수천 줄의 코드를 읽고 쓰는 무거운 작업은 codex, GLM과 같은 워커가 각자의 실행 한도와 컨텍스트에서 처리합니다. 서로 독립된 일을 동시에 실행하고 통합 비용을 관리하면 구현을 병렬로 진행할 수 있습니다. 이 책은 그 조건과 측정 방법을 함께 다룹니다.

2. 컨텍스트 순수성 유지. 지휘관 Claude의 세션에는 오직 '무엇을 할 것인가'와 워커가 요약해온 '결과'만 존재합니다. 중간 디버깅 과정의 쓰레기 텍스트가 쌓이지 않습니다. 워커들은 각자 독립된 프로세스와 컨텍스트를 가지므로, 한 워커가 환각(Hallucination)에 빠져도 전체 시스템에는 영향을 주지 않습니다. 워커가 실패하면 버리고 새 워커를 띄우면 그만입니다.

3. 통제 가능한 병렬 확장의 기반. 이 구조는 본질적으로 병렬적입니다. API 엔드포인트 수정, 프론트엔드 UI 변경, 데이터베이스 마이그레이션 스크립트 작성을 동시에 진행할 수 있습니다. 단, 같은 파일과 같은 상태를 건드리는 일까지 무작정 겹치면 충돌 수습 비용이 이득을 삼킵니다. 병렬화의 단위는 워커 수가 아니라 **독립적으로 검증 가능한 작업** 수입니다.

비교 항목	단일 세션	지휘관 + 워커 풀
판단과 구현	한 컨텍스트에 섞임	지휘관과 실행 컨텍스트가 분리됨
대기	한 작업이 끝나야 다음 작업 시작	독립 작업을 동시에 실행
실패 범위	세션 전체가 오염될 수 있음	실패한 작업 단위만 재시도
완료 근거	모델의 완료 문장에 의존하기 쉬움	프로세스·로그·테스트를 함께 확인
통합 비용	낮지만 처리량도 한 세션에 묶임	높아질 수 있어 명세와 검증 게이트가 필요

첫날부터 적용할 최소 운영 루프

지휘관은 작업을 쓰지 않고 완료 조건을 씁니다

[왜] “로그인 기능을 개선해 줘”처럼 넓은 요청을 그대로 워커에게 넘기면 워커는 질문을 하거나, 관련 없는 파일까지 고치거나, 테스트 없이 멈출 수 있습니다. 병렬 실행에서는 모호함이 워커 수만큼 복제됩니다.

[규칙] 첫 실행에서는 목표, 읽을 파일, 수정할 파일, 하지 않을 일, 검증 명령, 완료 보고 형식만 고정합니다. 독립 작업 하나는 3~15분 안에 결과를 판정할 수 있는 크기로 자릅니다. 아직 판정할 수 없다면 워커를 늘리지 말고 명세를 더 작게 만듭니다.

[명령/코드] 다음은 문서 한 파일만 점검하는 가장 작은 호출입니다. `-m`과 `-c`는 이 책과 함께 저장한 `codex exec --help` 출력에서 확인한 옵션입니다.

```
cd ~/project/myapp
mkdir -p docs/logs
codex exec -m gpt-5.6-luna -c model_reasoning_effort="low" \
  "TASK: README 설치 명령 점검. SCOPE: README.md만 수정. OUT-OF-SCOPE: 코드 수정 금지. VERIFY: README의 명령을
  셀 문법으로 검토. REPORT: 마지막 줄 DONE: README 점검" \
  < /dev/null > docs/logs/readme-worker.log 2>&1 &
worker_pid=$!
printf '%s\n' "$worker_pid" > docs/logs/readme-worker.pid
```

정상 로그의 마지막 부분은 다음처럼 **결론 본문**을 포함해야 합니다.

```
Verified: npm install, npm test
Changed: README.md의 오래된 스크립트명 1건
DONE: README 점검
tokens used
```

[실패 신호] 프로세스는 끝났지만 로그가 “계획을 승인해 주시면 진행하겠습니다”에서 멈추거나, `codex` 로그에 `tokens used`가 없습니다. 반대로 프로세스가 살아 있는데 로그 크기만 작다고 실패로 단정하는 것도 오판입니다.

[복구] 먼저 PID의 생존 여부와 로그 본문을 함께 확인합니다. 승인 대기라면 같은 명령을 그대로 반복하지 않습니다. 명세 앞에 “물어보지 말고 즉시 구현하라. 승인 대기 금지”를 추가하고, 빠진 완료 조건을 보완해 한 번만 변경 재시도합니다. 두 번째에도 같은 이유로 실패하면 다른 워커로 넘깁니다.

이 작은 루프만 지켜도 “끝난 프로세스”와 “끝난 작업”을 혼동하는 첫 사고를 크게 줄일 수 있습니다.

```
pid=$(cat docs/logs/readme-worker.pid)
kill -0 "$pid" 2>/dev/null && echo RUNNING || echo EXITED
tail -n 12 docs/logs/readme-worker.log
```

이 책의 구조와 읽는 법

이 책은 개념서가 아닙니다. 제가 직접 프로젝션을 굴리며 피눈물을 흘리며 깎아낸 CLAUDE.md 규칙과 Bash 스크립트의 모음입니다. 각 장은 철저하게 실전 중심으로 구성되어 있습니다.

모든 핵심 규칙은 다음의 흐름으로 전개됩니다:

1. **[왜]:** 왜 이 규칙이 필요한지 (주로 뼈아픈 실패 사례)
2. **[규칙]:** 우리가 지켜야 할 명확한 원칙
3. **[명령/코드]:** 실제로 복사해서 붙여넣을 수 있는 CLI 명령과 스크립트
4. **[실패 신호]:** 이 규칙이 어긋났을 때 나타나는 증상
5. **[복구]:** 실패했을 때 시스템을 정상화하는 방법

책을 처음부터 끝까지 정독할 필요는 없습니다. 당장 2장의 세팅을 마치고, 12장의 템플릿을 여러분의 프로젝트 루트에 복사해 넣으십시오. 그리고 Claude Code를 켜서 지휘를 시작하십시오. 막히는 부분이 생기면 그때 해당 챕터를 찾아 읽으면 됩니다.

주석 1: 본문에 등장하는 `gpt-5.6-sol`, `glm-5.3`, `claude-fable-5` 등의 모델명은 2026년 8월 기준입니다. 독자 여러분이 이 책을 읽는 시점에는 항상 최신 모델명으로 치환하여 사용하십시오.

예고: 피로 쓴 3건의 실제 사고 사례

이 책의 모든 규칙은 이론이 아니라 실제 사고를 겪으며 만들어졌습니다. 책 전반에 걸쳐 다음 3가지 중대한 사고의 전말과 해결책이 상세히 다뤄집니다.

- **2026-08-18 Stall 사고:** 워커가 완료 보고 없이 멈췄는데 지휘관이 완료로 착각해 몇 시간을 날린 사건. 워커를 어떻게 감시해야 하는지(5장)의 교훈이 되었습니다.
- **2026-08-19 오판 사고:** 비-codex 워커의 정상 종료를 에러로 오판하여 모든 작업이 멈춘 사건. 워커별 특성과 결과 본문 확인의 중요성(5장, 7장)을 일깨웠습니다.
- **2026-08-20 Chrome 프로필 소실 사고:** 브라우저 자동화가 사용자의 기본 프로필을 건드려 북마크 2천여 개가 사라진 치명적 사고. 안전한 격리 환경 구축(9장)의 절대적인 기준이 되었습니다.

우리는 이 사고들을 딛고 올라섰습니다. 목표는 첫날부터 완벽해지는 것이 아닙니다. 실패를 빨리 발견하고, 같은 실패가 다시 일어나지 않도록 규칙과 검사로 고정하는 것입니다. 이제 1장으로 넘어가 지휘관과 워커의 역할을 명확히 나누는 방법부터 살펴보겠습니다.

「체크리스트」

- [] 현재 단일 모델로 모든 코딩을 처리하며 한도 부족을 겪고 있는가?
- [] 에이전트가 코드를 작성하는 동안 화면만 바라보며 대기하고 있는가?
- [] 긴 세션 후 모델이 이전에 했던 지시를 잊거나 엉뚱한 코드를 고치는가?
- [] 복수의 파일이나 모듈을 동시에 수정해야 할 일이 잦은가?
- [] 백그라운드 프로세스와 터미널 환경(zsh/bash)에 익숙한가?

- [] 워커의 완료 문장과 실제 검증 통과를 구분할 준비가 되었는가?

「흔한 실패 3가지」

1. **여전히 Claude에게 코드를 짜라고 지시함**: 지휘관에게 망치를 쥐여주면 안 됩니다. 명세만 내리고 워커를 부르십시오.
2. **워커를 순차적으로 하나씩만 실행함**: 병렬성의 이점을 버리는 것입니다. 독립된 작업은 무조건 동시 실행하십시오.
3. **템플릿과 규칙을 무시하고 말로만 지시함**: 엄격한 마커(Marker)와 프로세스 관리가 없으면 워커들은 곧 통제 불능 상태에 빠집니다.

1. 역할 분리 설계: Claude는 지휘관, 나머지는 손

혼자 개발할 때 에이전트 하나에게 기획, 파일 탐색, 구현, 테스트, 리뷰, 커밋까지 전부 맡기면 처음에는 편해 보입니다. 그러나 긴 작업일수록 그 편의는 대기 시간, 컨텍스트 오염, 승인 대기, 놓친 검증으로 되 돌아옵니다. 이 장의 목적은 모델을 줄 세우는 일이 아닙니다. 한 세션이 판단을 붙들고, 여러 워커가 손발처럼 움직이게 만드는 운영 구조를 만드는 일입니다.

여기서 Claude는 지휘관입니다. 지휘관은 무엇을 만들지 결정하고, 작업을 작게 나누고, 결과가 정말 맞는지 확인합니다. codex, GLM, agy는 워커입니다. 워커는 정해진 범위 안에서 읽고, 고치고, 테스트하고, 결과를 냅니다. 사람이 하던 팀 리드와 실행자의 구분을 한 사람의 터미널 안에 옮기는 셈입니다.

이 구분은 권력의 문제가 아닙니다. 실패를 빨리 발견하고, 좋은 판단에 비싼 컨텍스트를 쓰며, 병렬로 끝낼 수 있는 일을 기다리지 않게 하는 문제입니다. 모델이 많아질수록 이 원칙이 없으면 오히려 결과가 나빠집니다.

1. 한 세션 만능주의를 버리는 이유

컨텍스트는 공용 작업대가 아닙니다

[왜] 하나의 세션이 저장소 전체를 탐색한 뒤 구현하고 테스트까지 하면, 앞에서 읽은 수많은 세부 사항이 뒤의 판단을 밀어냅니다. 작은 오류 메시지와 오래된 가설도 같은 작업대에 쌓입니다. 결국 핵심 요구보다 방금 본 로그에 반응하기 쉽습니다.

[규칙] 요구의 해석, 우선순위, 수용 기준, 최종 판정만 지휘관 컨텍스트에 둡니다. 파일 대량 탐색, 반복 수정, 긴 테스트 로그는 워커의 컨텍스트로 보냅니다.

[명령/코드] 다음처럼 지휘관은 작업 명세를 전달하고, 워커에게 필요한 파일만 읽게 합니다.

```
codex exec -m gpt-5.6-sol -c model_reasoning_effort="high" "TASK: 결제 상태 표시 오류를 수정한다. CONTEXT: src/billing/status.ts와 tests/billing/status.test.ts만 먼저 읽는다. SCOPE: 상태 변환과 테스트를 수정한다. OUT-OF-SCOPE: 다른 리팩터링은 하지 않는다. VERIFY: npm test -- status. REPORT: 마지막 줄에 DONE: <요약>을 출력한다. 물어보지 말고 즉시 구현하라. 승인 대기 금지. 불명확하면 합리적 기본값으로 진행하고 완료 후 가정을 보고하라."
```

[실패 신호] 지휘관이 같은 로그를 여러 번 읽거나, 워커가 무엇을 고쳤는지보다 저장소의 배경 설명만 길게 보고합니다. 또는 한 작업의 대화가 길어져 처음의 수용 기준을 다시 찾아야 합니다.

[복구] 현재 작업을 멈추고 목표, 수정 허용 파일, 검증 명령을 다섯 줄 안으로 다시 씁니다. 그 명세만 새 워커에게 전달합니다. 기존 대화의 장황한 설명을 통째로 붙이지 않습니다.

대기는 가장 비싼 유향 시간입니다

[왜] 에이전트가 테스트를 돌리는 동안 지휘관까지 기다리면, 사람 한 명이 아닌 모든 모델이 같은 병목에 묶입니다. 반대로 독립된 문서화, 구현, 조사, 테스트는 서로 기다릴 이유가 없습니다.

[규칙] 파일과 수용 기준이 겹치지 않는 작업은 동시에 분배합니다. 단, 같은 파일을 둘 이상이 직접 수정하게 해서는 안 됩니다. 충돌하는 작업은 순차화하거나 별도 worktree에서 비교합니다.

[명령/코드] 아래는 역할이 겹치지 않는 세 갈래를 보낸 예입니다. 실제 작업에서는 각 명세에 파일 범위를 더 좁게 적습니다.

```
codex exec -m gpt-5.6-sol -c model_reasoning_effort="high" "TASK: API 오류 처리 구현. CONTEXT: src/api/error.ts만 읽고 수정. VERIFY: npm test -- api-error. REPORT: DONE 또는 FAILED."
codex exec -m gpt-5.6-terra "TASK: 설정 문서 갱신. CONTEXT: docs/configuration.md만 수정. VERIFY: npm run lint:docs. REPORT: DONE 또는 FAILED."
glm -p --permission-mode acceptEdits "TASK: 단위 테스트 보강. CONTEXT: tests/api/error.test.ts만 수정. VERIFY: npm test -- api-error. REPORT: 마지막 줄 DONE 또는 FAILED."
```

[실패 신호] 여러 워커가 모두 실행 중인데 `git diff`에서 같은 파일이 계속 바뀌거나, 한 워커의 수정이 다른 워커의 테스트를 깨뜨립니다. 반대로 독립 항목이 세 개인데 한 워커만 일하고 나머지는 놀고 있습니다.

[복구] 즉시 파일 소유권 표를 만듭니다. 겹친 작업 하나를 중지하고, 변경을 비교할 필요가 있으면 `worktree`로 분리합니다. 유틸 워커에는 새 구현을 중복으로 주기보다 문서 검토, 테스트 보강, 적대적 리뷰처럼 독립적인 품질 작업을 줍니다.

완료 보고는 증거가 아닙니다

[왜] 워커는 계획을 말하고 멈출 수 있고, 테스트 실패를 놓칠 수 있으며, `DONE`이라는 한 줄을 잘못 출력할 수도 있습니다. 특히 2026-08-18에는 `codex` 워커가 승인 요청만 남기고 완료 마커 없이 종료했는데, 지휘관이 완료로 오해해 시간이 손실되었습니다.

[규칙] 지휘관은 보고 문장만으로 승인하지 않습니다. 결과 본문을 읽고, `diff`를 보고, 지정한 검증 명령의 종료 코드를 직접 확인한 뒤에만 완료로 판정합니다.

[명령/코드] 최종 확인은 긴 로그 대신 끝부분과 종료 코드를 함께 읽습니다.

```
(
  set -o pipefail
  npm test -- api-error 2>&1 | tail -n 5
  test_status=$?
  printf 'exit=%s\n' "$test_status"
  exit "$test_status"
)
git diff --stat
git diff -- src/api/error.ts tests/api/error.test.ts
```

[실패 신호] “테스트를 통과했습니다”라는 말은 있으나 명령, 변경 파일, 종료 코드가 없습니다. 또는 `diff`가 명세 범위 밖 파일까지 포함합니다.

[복구] 지휘관이 검증 명령을 한 번 실행합니다. 실패하면 결과를 붙여 워커에게 수정 루프를 돌려 달라고 요청합니다. 범위 밖 변경은 먼저 별도 패치로 보존한 뒤 현재 작업에서 분리하고, 왜 생겼는지 명세의 제외 범위를 보강합니다.

2. 왜 Claude 안의 서브에이전트가 아니라 외부 CLI 워커인가

Claude Code에는 세션 안에서 하위 에이전트를 띄우는 네이티브 기능(Agent/Task 도구)이 있습니다. "이미 병렬이 되는데 굳이 왜 외부 CLI인가"는 정당한 질문입니다. 답부터 말하면, 이 책이 노리는 것은 병렬 처리 자체가 아니라 **한도·지속성·실패 판정을 세션 밖에 두는 일**이기 때문입니다.

네이티브 서브에이전트와 외부 CLI 워커 비교

[왜] 네이티브 서브에이전트는 지휘관과 같은 Claude 구독 계정, 같은 세션 안에서 돕니다. 편리하지만 그만큼 지휘관의 한도와 운명을 공유합니다. 선택 기준 없이 쓰면 "병렬로 돌렸는데 왜 Claude 한도는 그대로 닳지?"라는 역효과가 생깁니다.

[규칙] 기본값은 외부 CLI 워커입니다. 네이티브 서브에이전트는 아래 표의 단점이 이번 작업에서 이득보다 작을 때에만 씁니다.

비교 항목	네이티브 서브에이전트(Agent/Task)	외부 CLI 워커(codex·GLM·agy)
한도 분산	안 됩니다 — 지휘관과 같은 Claude 구독 한도를 소모합니다	됩니다 — 각 워커의 구독·제공자 한도를 따로 소모합니다
모델 다양성	Claude 계열 모델 안에서 선택합니다	GPT 계열(codex)·GLM·Gemini 계열까지 제공자별 적성 배분이 가능합니다
컨텍스트 격리	탐색 부담은 서브에이전트가 지고 요약 보고만 반환됩니다. 단, 반환된 보고서는 지휘관 대화에 그대로 쌓입니다	완전히 별개 프로세스와 로그 파일입니다. 지휘관은 결론과 로그 꼬리만 골라 읽습니다
백그라운드 지속성	지휘관 세션 안에서 관리됩니다	별도 프로세스와 로그로 관리할 수 있습니다. 단, 터미널 종료 뒤 생존은 자동 보장되지 않으므로 감시 래퍼나 감독 프로세스가 필요합니다
실패 격리	실패가 세션 안 오류로 반환됩니다. 계정 한도가 소진되면 전원이 동시에 멈춥니다	프로세스 단위로 격리됩니다 — exit 코드와 로그로 개별 판정하며, 한 워커의 한도 소진은 다른 워커와 무관합니다
비용	전부 Claude 플랜 토큰입니다	각 워커의 플랜입니다. Claude 토큰은 명세 작성·결론 회신·검증에만 씁니다

[명령/코드] 외부 워커는 로그와 PID를 남겨 지휘관 대화와 상태를 분리합니다. 이 예시는 현재 셸이 유지되는 동안의 실행 패턴이며, 셸 종료 뒤 지속성을 보장하지 않습니다.

```
mkdir -p .logs
codex exec -m gpt-5.6-terra "<brief>" \
  < /dev/null > .logs/1420_terra_filter.log 2>&1 &
worker_pid=$!
printf '%s\n' "$worker_pid" > .logs/1420_terra_filter.pid
tail -n 20 .logs/1420_terra_filter.log
```

[실패 신호] 한도 절감을 기대하며 네이티브 서브에이전트로 구현을 병렬화했는데 Claude 주간 한도가 같은 속도로 닳습니다. 세션을 재시작했더니 실행 중이던 하위 작업의 흔적이 대화와 함께 사라집니다.

[복구] 읽기 전용 조회 외의 구현·탐색·리뷰는 외부 CLI 워크로 옮깁니다. 세션이 죽기 전이라면 로그 파일이 남는 외부 호출형으로 재실행합니다.

네이티브가 더 나은 경우 세 가지.

정직하게, 다음 세 경우는 네이티브 서브에이전트가 이 책의 구조보다 낫습니다.

1. **짧은 읽기 전용 조회.** "이 함수가 어디 정의되어 있지" 수준의 탐색은 프로세스를 띄우고 로그를 여는 비용이 이득보다 큼니다. 네이티브는 결과가 대화로 바로 돌아오므로 로그 파일 관리와 결과 본문 읽기 규율(5장)도 필요 없습니다.
2. **지휘관 대화의 맥락이 필요한 하위 작업.** 방금 세운 계획의 세부를 여러 곳에 적용하는 작업처럼 부모 세션의 문맥을 물려받아야 할 때 유리합니다. 외부 CLI는 그 맥락을 명세로 다시 써야 하고, 이 옮겨 적기에서 왜곡이 생깁니다.
3. **다른 구독이 아직 없을 때.** codex.z.ai 계정이 없거나 래퍼 세팅 전이라면 네이티브가 유일한 병렬 수단입니다. 소규모 탐색 동시 처리 정도는 네이티브로 충분하고, 계정 추가 비용과 래퍼 유지보수 부담이 없습니다.

두 세계는 대립하지도 않습니다. GLM 래퍼(glm -p)는 claude CLI의 헤드리스 호출을 다른 제공자로 연결해 외부 워크로 만든 사례입니다(2장). 네이티브는 일상 조회용으로, 외부 CLI는 구현·탐색·리뷰용으로 함께 쓰는 것이 이 책의 실제 운영입니다.

3. Claude가 직접 할 일과 위임할 일

지휘관에게 모든 일을 맡기지 않는다고 해서 Claude가 수동적인 비서가 되는 것은 아닙니다. 오히려 가장 비싼 판단을 Claude에 남깁니다. 다음 표는 시작점입니다. 프로젝트마다 바꿀 수 있지만, 바꾸려면 왜 예외인지 한 줄로 남기는 편이 안전합니다.

일의 종류	기본 담당	이유	지휘관이 확인할 증거
요구 재진술과 수용 기준	Claude	애매함을 한 번에 고정해야 합니다	목표·범위·제외·검증 명령
작업 분해와 파일 소유권	Claude	충돌과 의존성을 먼저 봐야 합니다	3~15분 단위 명세
대량 코드 탐색	agy 또는 GLM	지휘관 컨텍스트를 보호합니다	파일 목록과 근거 요약
고난도 디버깅	codex Sol	깊은 추론과 구현을 함께 요구합니다	재현·수정·테스트 결과
일반 기능 구현	codex Terra 또는 GLM	명확한 명세를 빠르게 실행합니다	diff와 검증 종료 코드
기계적 반복	codex Luna	리네임·보일러플레이트에 적합합니다	변경 목록과 lint 결과
문서화와 조사	agy	독립 보조 작업으로 보내기 좋습니다	출처·변경 파일·결론
최종 적대적 리뷰	codex Sol	구현자의 확증 편향을 줄입니다	버그·엣지·회귀·보안 점검
커밋과 배포 승인	Claude	전체 맥락과 책임을 한 곳에 둡니다	검증 기록과 배포 성공 마커

기획과 명세는 지휘관의 일입니다

[왜] 구현자는 주어진 문제를 가장 빨리 푸는 방향으로 움직입니다. 그러나 무엇을 만들지, 무엇을 만들지 않을지, 어떤 실패가 허용되지 않는지는 구현 중에 정하면 흔들립니다. 여러 워커가 각자 해석하면 결과는 합쳐지지 않습니다.

[규칙] 워커를 호출하기 전에 지휘관이 목표, 문맥, 범위, 제외, 완료 기준, 검증 명령, 보고 형식을 한 장의 brief에 고정합니다. 질문이 필요할 정도로 구현이 갈라지지 않는다면 합리적 기본값을 명시하고 진행합니다.

[명령/코드] 다음 형식은 특정 도구가 아니라 작업 계약입니다.

```
TASK: 사용자가 알림을 읽음으로 표시할 수 있게 한다.
CONTEXT: src/notifications/service.ts, tests/notifications/service.test.ts를 읽는다.
SCOPE: 읽음 상태 저장과 해당 단위 테스트만 구현한다.
OUT-OF-SCOPE: UI 변경, 데이터베이스 마이그레이션, 무관한 리팩터링은 하지 않는다.
DONE-CRITERIA: 같은 알림을 두 번 처리해도 안전하고, 기존 테스트가 통과한다.
VERIFY: npm test -- notifications
REPORT: 마지막 줄에 DONE: <변경과 검증 요약>. 실패면 FAILED: <이유>.
```

[실패 신호] 워커가 “어떤 UI를 원하십니까?” 또는 “먼저 계획을 승인해 주십시오”라고 묻고 끝냅니다. 이것은 친절함이 아니라 실행 계약의 빈칸입니다.

[복구] brief에 “물어보지 말고 즉시 구현하라. 승인 대기 금지. 불명확하면 합리적 기본값으로 진행하고 완료 후 가정을 보고하라”를 넣습니다. 다만 결제, 계정 생성, 데이터 삭제처럼 되돌릴 수 없는 행위는 이 문장으로 자동화하지 않습니다.

구현과 반복은 워커의 일입니다

[왜] 파일을 열고 오류를 재현하고 테스트를 반복하는 일은 집중력보다 실행 시간이 큼니다. 지휘관이 직접 하면 다음 작업을 설계할 시간이 사라집니다. 워커가 스스로 수정-테스트 루프를 끝내야 병렬 구조가 유지됩니다.

[규칙] 워커에게는 수정 가능한 파일과 자체 검증 명령을 함께 줍니다. 지휘관은 매 실패마다 직접 고치지 않고, 실패 로그와 범위를 포함해 같은 워커에 최대 두 번까지만 재시도시킵니다.

[명령/코드] codex의 모델과 권한은 호출마다 눈에 보이게 적는 편이 재현에 좋습니다.

```
codex exec -m gpt-5.6-terra "TASK: 검색 필터의 빈 값 처리를 수정한다. CONTEXT: src/search/filter.ts와 tests/search/filter.test.ts. SCOPE: 빈 문자열과 공백만 처리. OUT-OF-SCOPE: API 형식 변경 금지. VERIFY: npm test -- filter. REPORT: 마지막 줄 DONE 또는 FAILED."
```

[실패 신호] 워커가 테스트 하나가 실패한 뒤 더 이상 시도하지 않거나, 같은 오류를 두 번 반복합니다. 또는 작업 중간에 다른 디렉터리까지 포괄적으로 고칩니다.

[복구] 첫 실패에는 오류 전문의 필요한 부분과 예상 동작을 brief에 추가해 재시도합니다. 두 번째 실패에는 다른 적성의 워커로 바꾸고, 이전 실패 원인을 한 줄로 전달합니다. 같은 입력을 그대로 세 번째 호출하지 않습니다.

최종 판단과 커밋은 지휘관의 일입니다

[왜] 여러 워커의 결과는 부분적으로 맞아도 전체적으로 모순될 수 있습니다. 한 워커는 테스트를 늘리고, 다른 워커는 API 계약을 바꿨을 수 있습니다. 커밋은 그 조합을 제품의 역사로 남기는 행위이므로 한 명의 판단자가 필요합니다.

[규칙] 지휘관은 변경 범위, 테스트 종료 코드, 리뷰 결과를 확인한 뒤에만 커밋 또는 배포 판단을 합니다. 워커가 커밋 메시지까지 제안할 수는 있지만 승인 권한은 갖지 않습니다.

[명령/코드] 비자명한 변경에는 독립 리뷰를 붙입니다.

```
codex exec -m gpt-5.6-sol -c model_reasoning_effort="high" "현재 변경을 적대적으로 리뷰하라. 버그, 오타, 케이스, 회귀, 보안을 확인한다. 발견한 문제는 직접 수정하고 npm test -- notifications를 실행한다. 마지막 줄에 DONE: <검토와 검증 요약> 또는 FAILED: <이유>를 출력한다."
```

[실패 신호] 리뷰가 구현자의 요약을 반복하거나, 테스트가 통과했다는 말만 하고 변경의 부작용을 검사하지 않습니다.

[복구] 리뷰 prompt에 확인할 파일, 변경 의도, 금지된 동작을 적습니다. 리뷰어가 같은 파일을 과도하게 바꾸면 수정 권한을 분리하거나, 발견 사항만 보고하게 한 뒤 지휘관이 별도 구현 워커에 배정합니다.

4. 워커 풀은 우선순위 체인이 아닙니다

“Sol이 실패하면 Terra, Terra가 실패하면 GLM” 같은 줄 세우기는 편해 보이지만 좋은 운영이 아닙니다. 그것은 한 워커가 일하는 동안 다른 워커를 의도적으로 놀리는 방식입니다. 이 책의 풀은 예비군 목록이 아니라 적성 지도입니다.

적성 배분 표를 먼저 만듭니다

[왜] 같은 기능 구현이라도 어떤 것은 설계 판단이 많고, 어떤 것은 이름 바꾸기와 형식 정리가 대부분입니다. 모델의 이름이나 막연한 체감 한도를 기준으로 배분하면 과한 모델을 단순 작업에 쓰거나 어려운 오류를 약한 문맥에 던지게 됩니다.

[규칙] 작업의 난이도, 파일 충돌 가능성, 반복성, 독립성을 보고 배정합니다. 한도를 미리 추측하거나 “이번 주에는 이 모델만 쓴다”라고 고정하지 않습니다. 실제 limit 오류가 났을 때만 해당 작업을 옮깁니다.

[명령/코드] 아래 표의 호출은 이 환경에서 확인한 인터페이스에 맞춘 예시입니다. 프로젝트 경로와 brief는 독자가 바꿉니다.

워커	적성	호출 예시	피할 배정
codex Sol	고난도 디버깅, 아키텍처가 얽힌 구현, 적대적 리뷰	<code>codex exec -m gpt-5.6-sol -c model_reasoning_effort="high" "<brief>"</code>	단순 파일명 일괄 변경만 맡기기
codex Terra	명확한 일반 구현, 테스트 수정	<code>codex exec -m gpt-5.6-terra "<brief>"</code>	다른 워커와 같은 파일 동시 수정
codex Luna	보일러플레이트, lint, 기계적 반복	<code>codex exec -m gpt-5.6-luna "<brief>"</code>	설계 선택이 많은 기능 전체

워커	적성	호출 예시	피할 배경
GLM	넓은 구현, 긴 문맥의 대량 작업	<code>glm -p --permission-mode acceptEdits "
<brief>"</code>	종량제 endpoint를 직접 부르는 호출
agy	조사, 코드 탐색, 문서화, 보조 리뷰	<code>agy --dangerously-skip-permissions -p "
<brief>"</code>	결론 없는 광범위한 저장소 순회

[실패 신호] 어려운 버그를 Luna에 계속 재시도하거나, 문서 한 파일을 위해 Sol을 기다립니다. 또는 GLM에서 모델 이름 경고가 보였다는 이유만으로 정상 결과를 버립니다.

[복구] 실패 원인을 적성 불일치, 명세 부족, 환경 오류로 나눕니다. 적성 불일치면 작업만 다른 워커에 옮깁니다. GLM의 `[claude-code:unrecognized_model]` 경고는 응답과 종료 코드가 정상이라면 정보성 경고이므로, 그 문자열만으로 실패 처리하지 않습니다.

유휴 금지는 무작정 많이 부리라는 뜻이 아닙니다

[왜] 유휴 워커를 없애겠다는 말은 같은 코드에 다섯 명을 투입하라는 뜻으로 오해되기 쉽습니다. 그런 병렬성은 충돌, 중복 비용, 비교 부담만 늘립니다.

[규칙] 유휴 금지는 독립적인 가치가 있는 일이 있을 때 적용합니다. 구현이 하나뿐이면 나머지 워커에는 테스트 설계, 문서 정확성 검토, 변경 영향 탐색, 적대적 리뷰처럼 파일 소유권이 분리된 일을 배정합니다.

[명령/코드] 한 기능 구현과 병행할 수 있는 탐색 작업은 읽기 범위를 명시해 보냅니다.

```
agy --dangerously-skip-permissions -p "TASK: 알림 읽음 상태 변경의 영향 범위를 조사한다. CONTEXT: src/notifications와 docs/api.md만 읽는다. SCOPE: 호출 지점, API 계약, 회귀 위험을 표로 보고한다. OUT-OF-SCOPE: 파일 수정 금지. DONE-CRITERIA: 영향 파일과 근거가 있다. REPORT: 마지막 줄 DONE: <요약>."
```

[실패 신호] 같은 패치가 두 워커에서 거의 똑같이 나오는데 채택 기준이 없습니다. 또는 결과를 비교할 시간보다 워커를 새로 띄우는 시간이 더 많습니다.

[복구] 이중 투입은 중요하고 고난도인 변경에만 씁니다. 두 구현을 비교할 때는 반드시 분리된 worktree를 사용하고, 정확성·테스트·변경 범위·유지보수성이라는 같은 기준으로 한 쪽을 채택합니다.

한도는 관측값이지 계획값이 아닙니다

[왜] “이 모델은 이번 주에 남았을 것”이라는 추측은 실제 사용량, 계정 상태, 제공자 정책을 모르는 상태에서 나온 이야기입니다. 추측을 기준으로 배분하면 좋은 워커가 놓고, 오류가 날 때까지 무의미한 우회가 생깁니다.

[규칙] 호출 전에는 적성으로 배분하고, 호출 후 실제 limit:rate limit:인증 오류가 확인될 때만 재배정합니다. 모델별 한도 숫자를 팀 규칙에 하드코딩하지 않습니다.

[명령/코드] 실패 내용을 보존한 간결한 재배정 명세입니다.

```
TASK: 이전과 같은 검색 필터 수정.
CONTEXT: 이전 워커는 rate limit으로 실행하지 못했다. src/search/filter.ts와 테스트 파일만 읽는다.
SCOPE: 기존 수용 기준을 그대로 구현한다.
OUT-OF-SCOPE: 이전 워커의 미완성 변경을 추측해 덮어쓰지 않는다.
VERIFY: npm test -- filter
```

REPORT: 마지막 줄 DONE 또는 FAILED.

[실패 신호] 오류가 없는데도 배분을 자주 바꾸거나, 경고 한 줄만 보고 동일 작업을 여러 워커에 중복 배정합니다.

[복구] 프로세스의 응답, 종료 코드, 결과 본문을 확인합니다. 워커 실패는 무응답 또는 `exit#0`으로 판정합니다. 경고 문자열, 로그 줄 수, 모델 이름만으로 실패로 단정하지 않습니다.

5. 지휘관의 5줄 규칙

복잡한 운영 규칙도 매 작업에서 기억하기 어렵습니다. 그래서 지휘관은 다음 다섯 줄을 매번 통과시킵니다. 이 규칙은 모델을 고르는 체크리스트가 아니라, 실행 가능한 작업인지 판정하는 최소 계약입니다.

1. 목표는 한 문장인가?
2. 수정하거나 읽을 파일 범위가 정해졌는가?
3. 하지 않을 일이 적혀 있는가?
4. 완료를 증명할 검증 명령이 있는가?
5. 종료 보고가 DONE 또는 FAILED로 정해졌는가?

다섯 줄이 없으면 실행하지 않습니다

[왜] 명세가 빈약하면 워커는 질문하거나, 자기 판단으로 범위를 넓히거나, 계획만 말하고 끝냅니다. 이런 결과를 모델의 성능 문제라고 부르면 같은 사고가 반복됩니다.

[규칙] 다섯 줄 중 하나라도 비면 먼저 brief를 보완합니다. 단, 오타 하나처럼 다섯 줄 이하의 명백한 수정은 지휘관이 직접 처리할 수 있습니다.

[명령/코드] 다섯 줄을 실제 prompt에 압축하면 다음과 같습니다.

```
TASK: 빈 검색어에서 서버 오류가 나지 않게 한다.
CONTEXT: src/search/filter.ts, tests/search/filter.test.ts.
SCOPE/OUT-OF-SCOPE: 빈 문자열 · 공백만 처리하고 API 형식은 바꾸지 않는다.
VERIFY: npm test -- filter.
REPORT: 마지막 줄 DONE: <요약> 또는 FAILED: <이유>. 즉시 구현하고 승인 대기하지 않는다.
```

[실패 신호] “완료”라는 말은 있는데 무엇을 검증했는지 없거나, 작업자가 파일을 찾느라 저장소 전체를 읽습니다.

[복구] 요구를 더 길게 쓰기보다 파일명과 검증 명령을 더 구체적으로 씁니다. 범위를 모르겠다면 탐색 전용 워커를 먼저 배정해 파일 목록만 가져옵니다.

6. 완료와 재배정은 별도 운영 규칙입니다

역할 설계 단계에서는 두 경계만 기억합니다. codex는 종료 뒤 `tokens used`와 결론 본문을 확인하고, Claude·GLM·agy는 프로세스 종료와 결론 본문을 확인합니다. 실패한 작업은 원인을 바꾼 한 번의 재시도 뒤 다른 적성의 워커로 옮깁니다. 상세 상태 전이와 사고 분석은 → 5장, 재배정 절차는 → 7장을 참고합니다.

7. 안전한 지휘 범위

2026-08-20 Chrome 프로필 사고가 보여 주듯 계정·결제·KYC·기본 브라우저 프로필은 워커 권한 밖입니다. 브라우저는 격리 프로필만 사용하고 작업 뒤 종료하며, 비밀값은 존재 여부만 확인합니다. 구체적인 금지 목록과 복구 절차는 → 9장을 참고합니다.

모델 이름은 계약의 일부입니다

[왜] 모델명이 바뀌거나 기본값이 바뀌면 같은 명세도 다른 성격의 실행이 됩니다. 특히 낮은 등급 모델이 조용히 대체되면 문제를 재현하기 어렵습니다.

[규칙] 호출에는 필요한 모델 ID와 권한을 명시합니다. `gpt-5.6-sol/terra/luna`, `glm-5.3`, `claude-fable-5/opus-5/sonnet-5` 같은 이름은 실행 당시의 계약으로 기록하고, 래퍼의 기본값에만 의존하지 않습니다.

[명령/코드] GLM은 제공된 래퍼를 통해 호출합니다. 래퍼는 `ANTHROPIC_BASE_URL`, 인증 토큰 전달, `CLAUDE_CODE_MAX_CONTEXT_TOKENS`를 프로세스 범위에서 설정합니다. 키 자체는 절대로 본문이나 저장소에 쓰지 않습니다.

```
GLM_MODEL=glm-5.3 glm -p --permission-mode acceptEdits "<brief>"
```

[실패 신호] 누군가 모델명을 추측해 바꾸거나, GLM의 정상 경고를 고치겠다며 설정 파일에 모델 매핑을 추가합니다.

[복구] 실제 래퍼의 출력과 종료 코드를 먼저 봅니다. `unrecognized_model` 경고만으로 정상 호출을 중단하지 않습니다. `settings.json`의 `modelOverrides` 같은 엔터프라이즈용 매핑으로 문제를 덮지 않습니다.

8. 첫날에 만들 운영 보드

지휘관 구조는 거대한 대시보드가 없어도 시작할 수 있습니다. 텍스트 표 하나면 충분합니다. 중요한 것은 모든 작업이 “실행 중”과 “결과 읽음”을 분리해 갖는 일입니다. 전자는 프로세스 상태이고, 후자는 지휘관의 책임 상태입니다.

ID	목표	담당	파일 소유권	검증	상태	결과 읽음
A	API 오류 수정	Sol	<code>src/api/error.ts</code>	<code>npm test -- api-error</code>	진행	아니 오
B	테스트 보강	GLM	<code>tests/api/error.test.ts</code>	같은 테스트	대기	아니 오
C	계약 조사	agy	읽기 전용	보고서	진행	아니 오
D	문서 갱신	Luna	<code>docs/api.md</code>	문서 lint	대기	아니 오

[왜] 이 표가 없으면 지휘관은 마지막으로 본 로그만 기억합니다. 먼저 끝난 작업을 놓치고, 이미 끝난 워커를 기다리거나, 같은 파일에 새 작업을 넣게 됩니다.

[규칙] 워커를 시작할 때 한 행을 만들고, 종료했을 때 결과 읽음 칸을 갱신합니다. 사용자에게 진행을 알리기 전에는 모든 시작한 행의 결과 본문을 읽습니다.

[명령/코드] 표는 Markdown 파일, 이슈, 메모 어디에 두어도 됩니다. 다만 상태 이름은 대기, 진행, 검증, 완료, 실패처럼 적게 유지합니다.

[실패 신호] “누가 무엇을 하고 있지?”를 로그를 뒤져서 알아내야 합니다.

[복구] 새 도구를 만들지 말고, 현재 작업만 표로 복원합니다. 실행 중인 프로세스, 종료한 프로세스, 읽지 않은 결과를 각각 한 줄로 적으면 충분합니다.

체크리스트

- Claude가 목표, 범위, 제외, 검증, 승인 판단을 말도록 분리했는가?
- 각 워커 brief에 파일 범위와 DONE 또는 FAILED 보고 형식이 있는가?
- 독립 작업만 병렬로 배정하고 같은 파일의 동시 수정을 막았는가?
- 배분을 한도 추측이 아니라 작업 적성으로 결정했는가?
- 네이티브 서브에이전트를 쓰기로 했다면, 그 이유가 한도 분산 착각이 아닌지 확인했는가?
- codex의 tokens used를 비-codex 워커의 완료 기준으로 오용하지 않았는가?
- 워커의 결과 본문, diff, 검증 명령 종료 코드를 모두 읽었는가?
- 계정·결제·기본 브라우저 프로필·비밀값을 워커 범위에서 제외했는가?

흔한 실패 3가지

1. 지휘관이 구현까지 떠안습니다. 다음 작업을 설계할 여유가 사라집니다. 파일 수정과 테스트 루프는 brief를 갖춘 워커로 보냅니다.

>

2. 모든 워커에 같은 패치를 시킵니다. 비교 기준 없는 중복은 병렬성이 아닙니다. 파일 소유권을 나누거나 중요한 변경만 분리된 worktree에서 이중 투입합니다.

>

3. 종료 메시지를 완료로 믿습니다. 보고는 증거가 아닙니다. 결과 본문, diff, 검증 종료 코드가 모두 있어야 완료입니다.

2. 30분 세팅: 워커 5기 설치·인증·래퍼

이 장에서는 지휘관 하나와 실행 워커 다섯 종류를 실제로 호출할 수 있는 최소 환경을 만듭니다. 목표는 멋진 대시보드가 아닙니다. 각 워커가 짧은 smoke test를 받고, 끝난 결과를 돌려주는지 확인하는 것입니다. 설치가 끝나도 작업을 시작하기 전에는 반드시 현재 CLI 도움말을 다시 확인하십시오. 도구의 기본값과 모델 목록은 바뀔 수 있습니다.

이 책의 예시는 비밀값을 절대 출력하지 않습니다. 실제 값은 셸 시작 파일이나 제공자의 안전한 자격 증명 방식으로 관리하고, 저장소·문서·작업 prompt에 넣지 않습니다.

1. 세팅 전에 정할 경계

작업 디렉터리를 먼저 고정합니다

[왜] 헤드리스 워커는 현재 디렉터리를 프로젝트로 해석합니다. 엉뚱한 상위 폴더에서 실행하면 파일을 찾지 못하거나, 의도하지 않은 저장소를 읽고 수정할 수 있습니다. 특히 codex의 비대화형 호출은 Git 저장소 여부에 영향을 받을 수 있습니다.

[규칙] 워커마다 프로젝트 루트로 먼저 이동해 실행합니다. 결과 파일과 로그도 그 프로젝트 내부에 둡니다. 이 책과 함께 제공된 `codex exec` 도움말에서 확인되지 않은 작업 디렉터리·Git 검사 생략 옵션은 예시에 넣지 않습니다.

[명령/코드] 다음은 현재 작업 위치를 확인하는 가장 작은 준비입니다.

```
pwd
git rev-parse --show-toplevel
```

프로젝트 루트는 `cd`로 명시합니다. 이 방식은 별도 작업 디렉터리 옵션에 의존하지 않습니다.

```
PROJECT_ROOT=$(git rev-parse --show-toplevel) || exit 1
cd "$PROJECT_ROOT" || exit 1
codex exec -m gpt-5.6-luna "TASK: 현재 폴더의 Markdown 제목을 점검한다. CONTEXT: Markdown 파일만 읽는다. SCOPE: 수정하지 않고 문제 목록만 낸다. VERIFY: 없음. REPORT: 마지막 줄 DONE: <요약>."
```

[실패 신호] 워커가 프로젝트와 무관한 파일을 언급하거나, Git 저장소 확인 오류 때문에 실제 작업을 시작하지 못합니다.

[복구] 새 터미널에서 `pwd`를 확인하고 프로젝트 루트로 이동합니다. Git 저장소가 필요한 작업이라면 `git rev-parse --show-toplevel`이 성공하는 위치에서 다시 실행합니다. 로컬 `codex exec --help`와 제공된 도움말에 모두 없는 옵션을 추측해 붙이지 않습니다.

비밀값은 존재만 검사합니다

[왜] API 키와 토큰은 로그, 셸 히스토리, 작업 prompt, 스크린샷에 한 번만 섞여도 회수하기 어렵습니다. 워커는 문제 해결을 위해 입력을 요약하거나 파일을 읽을 수 있으므로, 키가 들어간 파일을 문맥으로 넘기는 것도 위험합니다.

[규칙] 인증 문제는 값 대신 `SET` 또는 `missing`으로만 확인합니다. 키는 저장소의 `.env`, 문서, 코드 블록에 쓰지 않습니다. 셸 시작 파일에 넣을 때도 공개 저장소와 분리된 개인 파일을 사용합니다.

[명령/코드] 아래 명령은 값 자체를 출력하지 않습니다.

```
if [ -n "$ZAI_API_KEY" ]; then
  echo "ZAI_API_KEY=SET"
else
  echo "ZAI_API_KEY=missing"
fi
```

[실패 신호] `echo $ZAI_API_KEY`, `env`, `printenv`처럼 전체 값을 출력하는 명령을 실행하려 하거나, 키를 고정 문자열로 설정 파일에 붙여 넣습니다.

[복구] 즉시 해당 출력을 닫고 공유하지 않습니다. 노출 가능성이 있으면 제공자에서 키를 폐기·교체합니다. 이후에는 존재 확인 명령만 실행하고, 워커의 CONTEXT에 `.env`를 넣지 않습니다.

2. 실제 설치와 구독 경계

네 가지 도구를 설치하고 버전으로 확인합니다

[왜] 이 책의 워커 다섯 종류는 실행 파일 네 개로 움직입니다. `codex CLI`(Sol·Terra·Luna는 같은 실행 파일에 모델만 다름), `Claude Code`(지휘관이자 GLM 래퍼의 본체), `Gemini CLI` 또는 `Antigravity CLI`(`agy`)입니다. 설치 명령은 바뀔 수 있으므로, 아래 명령은 이 책을 쓴 시점(2026-08)에 저자 환경에서 실제로 설치되어 있던 버전과 함께 적습니다.

[규칙] 설치 후에는 반드시 `--version`으로 실행 파일이 응답하는지 확인하고, 그 다음 절의 smoke test로 "prompt를 받아 결과를 돌려주는지"까지 확인합니다. 패키지명·로그인 절차가 아래와 다르면 각 제공자의 현재 공식 문서가 우선입니다.

[명령/코드]

```
# 1) codex CLI — OpenAI. ChatGPT 구독(Plus/Pro/Team 등) 로그인으로 사용
npm install -g @openai/codex
codex login          # 브라우저가 열리며 ChatGPT 계정으로 로그인
codex --version      # 저자 환경: codex-cli 0.147.0

# 2) Claude Code — Anthropic. Claude Pro/Max 구독 또는 API 키
# 공식 설치 스크립트(권장) 또는 npm 중 하나
curl -fsSL https://claude.ai/install.sh | bash
# 또는: npm install -g @anthropic-ai/claude-code
claude --version     # 저자 환경: 2.1.251 (Claude Code)
claude               # 첫 실행 시 로그인 안내
```

```
# 3) Gemini CLI — Google. 개인 Google 계정 로그인으로 무료 한도 제공
npm install -g @google/gemini-cli
gemini --version      # 저자 환경: 0.51.0

# 4) agy — Google Antigravity CLI(Gemini 계열). 이 책에서 조사·탐색 워커로 사용
# 설치·로그인은 Antigravity 공식 안내를 따릅니다. 없으면 3)의 gemini 로 대체 가능(5절 참고)
agy --version         # 저자 환경: 1.1.22
```

도구	실행 파일	계정/구독	이 책에서의 역할
codex CLI	<code>codex</code>	ChatGPT 구독 로그인	워커 Sol/Terra/Luna (<code>-m</code> 으로 모델 선택)
Claude Code	<code>claude</code>	Claude 구독	지휘관 + GLM 래퍼의 본체
GLM 래퍼	<code>glm</code> (구매자 파일 <code>templates/glm_wrapper.sh</code>)	z.ai 코딩 플랜 구독 + <code>ZAI_API_KEY</code>	대량 병렬 구현 워커
Gemini CLI / agy	<code>gemini</code> / <code>agy</code>	Google 계정	조사·탐색·문서화 워커

GLM 래퍼는 별도 설치가 아니라 **Claude Code**를 **z.ai endpoint**로 돌리는 **셸 스크립트**입니다. 구매자 파일의 `templates/glm_wrapper.sh`를 `~/.local/bin/glm`처럼 PATH 안에 복사하고 실행 권한을 주면 됩니다. 전문은 4절과 12장에 있습니다.

```
cp templates/glm_wrapper.sh ~/.local/bin/glm && chmod +x ~/.local/bin/glm
```

[실패 신호] `--version`은 응답하는데 smoke test가 로그인 화면이나 권한 오류로 끝납니다. 또는 `codex --version`은 되는데 `codex exec`가 "Not inside a trusted directory" 로 즉시 종료합니다(3절의 `--skip-git-repo-check -C <경로>` 참고).

[복구] 실행 파일 존재(`command -v`), 버전 응답, 로그인, smoke test를 네 단계로 나눠 어디서 끊기는지 기록합니다. 인증 문제는 각 도구의 `login`/첫 실행 안내로 돌아가고, 책의 명령을 고치기 전에 현재 `--help` 출력을 먼저 확인합니다.

실행 파일 존재만 먼저 확인합니다

[명령/코드] 값이나 계정명을 출력하지 않고 실행 파일 존재만 확인합니다.

```
for worker_command in codex claude glm agy; do
  if command -v "$worker_command" >/dev/null 2>&1; then
    printf '%s=FOUND\n' "$worker_command"
  else
    printf '%s=missing\n' "$worker_command"
  fi
done
```

FOUND는 "설치됨"일 뿐 인증·모델 권한·구독 준비 완료가 아닙니다. 그 판정은 smoke test의 종료 코드와 결론 본문으로만 합니다.

3. codex CLI: Sol·Terra·Luna를 분리해 호출하기

codex는 이 구조에서 구현과 리뷰의 핵심 워커입니다. 동일한 `codex exec` 명령을 쓰되, 작업 난이도에 따라 모델과 추론 노력을 명시합니다. 기본 설정을 믿고 호출하면 다른 셸이나 다른 기기에서 기대와 다른 effort가 적용될 수 있습니다.

`gpt-5.6-sol`, `gpt-5.6-terra`, `gpt-5.6-luna`는 별도 실행 파일이 아니라 `codex exec -m`에 넘기는 모델 ID입니다. 이 책의 모델명 기준 시점은 0장의 각주에서 한 번만 밝힙니다.

비대화형 호출의 기본 형태

[왜] 대화형 창은 사람이 답할 때는 좋지만, 여러 작업을 동시에 보내고 로그로 수거할 때는 입력을 기다릴 수 있습니다. `codex exec`는 prompt 하나를 받은 뒤 비대화형으로 실행하는 형태라 워커 풀에 맞습니다.

[규칙] 작업마다 `-m`으로 모델을 지정합니다. Sol에는 `-c model_reasoning_effort="high"`를 명시해 기본 effort의 차이를 제거합니다. 샌드박스나 쓰기 권한은 이 책의 잘린 도움말로 확인되지 않은 플래그를 복사하지 말고, 현재 설치의 도움말과 프로젝트 정책에서 별도로 확인합니다.

[명령/코드] 어려운 오류 분석과 수정에는 Sol을 사용합니다.

```
codex exec -m gpt-5.6-sol -c model_reasoning_effort="high" "TASK: 날짜 경계 오류를 수정한다. CONTEXT: src/date/range.ts와 tests/date/range.test.ts만 읽는다. SCOPE: 시간대 경계와 단위 테스트를 수정한다. OUT-OF-SCOPE: 의존성 업데이트 금지. VERIFY: npm test -- date-range. REPORT: 마지막 줄 DONE: <변경 · 검증 요약> 또는 FAILED: <이유>. 물어보지 말고 즉시 구현하라. 승인 대기 금지. 불명확하면 합리적 기본값으로 진행하고 완료 후 가정을 보고하라."
```

[실패 신호] Sol이 느리다는 이유만으로 모델을 생략하거나, 기본 effort가 낮게 적용되어 복잡한 오류에서 알은 수정만 나옵니다.

[복구] 실행 로그가 아니라 호출 명령을 먼저 봅니다. `-m gpt-5.6-sol`과 `-c model_reasoning_effort="high"`가 있었는지 확인합니다. 명세가 충분한데도 두 번 실패하면 Terra를 반복 호출하지 말고, 실패 근거를 붙여 다른 적성의 워커로 옮깁니다.

Terra와 Luna는 보조가 아니라 다른 작업대입니다

[왜] Sol만 쓰면 다른 독립 작업이 대기열에 쌓입니다. Terra는 명확한 일반 구현을, Luna는 형식적이고 반복적인 변경을 맡으면 지휘관이 병목을 만들지 않습니다.

[규칙] Terra에는 기능 하나와 테스트 하나처럼 경계가 분명한 일을 줍니다. Luna에는 문서 링크 정리, 보일러 플레이트, lint 수정처럼 설계 결정이 작은 일을 줍니다. 둘 다 같은 파일을 Sol과 동시에 수정하지 않습니다.

[명령/코드] 일반 구현과 기계적 점검의 예시는 다음과 같습니다.

```
codex exec -m gpt-5.6-terra "TASK: 입력 검증 메시지를 추가한다. CONTEXT: src/forms/validate.ts와 tests/forms/validate.test.ts. SCOPE: 빈 이메일 오류만 추가한다. OUT-OF-SCOPE: UI 문구 전면 개편 금지. VERIFY: npm test -- validate. REPORT: 마지막 줄 DONE 또는 FAILED."
```

```
codex exec -m gpt-5.6-luna "TASK: 문서의 오래된 명령 이름을 교체한다. CONTEXT: docs/cli.md만 수정한다. SCOPE: old-command를 new-command로 바꾼다. OUT-OF-SCOPE: 예제 구조 변경 금지. VERIFY: npm run lint:docs. REPORT: 마지막 줄 DONE 또는 FAILED."
```

[실패 신호] Luna가 설계 결정을 묻고 멈추거나, Terra가 저장소 전반의 아키텍처를 재구성합니다.

[복구] Luna의 작업을 더 기계적으로 쪼갭니다. Terra의 범위를 파일 단위로 줄입니다. 설계 결정을 요구하는 조각은 Sol 또는 지휘관의 명세 단계로 되돌립니다.

config.toml의 기본 effort를 점검합니다

[왜] `~/codex/config.toml`의 기본 effort는 편의를 위해 설정되어 있을 수 있지만, 워커 풀에서 기대하는 품질 수준과 다를 수 있습니다. 특히 복잡한 작업은 호출자가 effort를 명시하지 않으면 환경별 결과가 흔들립니다.

[규칙] config.toml은 개인 기본값으로만 취급합니다. 중요한 Sol 호출에는 항상 `-c model_reasoning_effort="high"` 또는 필요한 값으로 덮어씁니다. 전역 기본값을 바꾸기 전에 기존 자동화가 어떤 값을 기대하는지 확인합니다.

[명령/코드] 비밀값이 들어 있을 수 있으므로 전체 파일을 공유하지 않고, 관련 키만 찾습니다.

```
rg -n "model_reasoning_effort|model" ~/.codex/config.toml
```

[실패 신호] “내 컴퓨터에서는 잘 되었다”는 말뿐이고, 어떤 모델·effort로 실행했는지 재현할 수 없습니다.

[복구] 중요한 명령에 모델과 effort를 적습니다. config.toml 수정은 여러 프로젝트에 영향을 줄 수 있으므로, 한 프로젝트의 특수 요구를 전역 기본값으로 바꾸지 않습니다.

codex smoke test

[왜] 설치 여부는 명령 이름이 존재한다는 사실보다, 선택한 모델이 비대화형 prompt를 받고 결과를 끝까지 반환하는지로 확인해야 합니다.

[규칙] smoke test는 파일을 수정하지 않는 짧은 읽기 작업으로 합니다. 실제 제품 코드나 비밀 파일을 문맥으로 주지 않습니다.

[명령/코드] Git 저장소 안에서는 다음처럼 실행합니다.

```
codex exec -m gpt-5.6-luna "TASK: 현재 프로젝트의 최상위 파일 이름 세 개만 보고한다. CONTEXT: 최상위 디렉터리만 읽는다. SCOPE: 파일 수정 금지. VERIFY: 없음. REPORT: 마지막 줄 DONE: <파일 목록 요약>."
```

[실패 신호] 프로세스가 종료했는데 codex 로그에 `tokens used`가 없거나, 결과 본문 없이 승인 요청만 남습니다.

[복구] prompt 끝에 즉시 실행과 `DONE` 형식을 다시 넣고 한 번 재시도합니다. 다시 실패하면 오류와 현재 디렉터리를 기록하고, 다른 워커로 작업을 재배정합니다. 단순히 성공으로 가정하지 않습니다.

4. Claude Code 헤드리스 워커와 GLM 래퍼

Claude Code는 -p로 한 번에 실행합니다

[왜] Claude Code를 워커로 쓸 때 대화형 창을 열면 다음 입력을 기다릴 수 있습니다. 병렬 작업에서는 prompt를 인자로 넘겨 종료까지 한 번에 받는 방식이 더 예측 가능합니다.

[규칙] 헤드리스 호출에는 -p를 사용합니다. 수정 작업에는 --permission-mode acceptEdits를 명시합니다. 명령 실행 권한까지 자동으로 넓히는 설정은 신뢰하는 프로젝트에서만 제한적으로 사용하고, 파일 범위를 먼저 줄입니다.

[명령/코드] 읽기 전용 smoke test는 다음처럼 짧게 합니다.

```
claude -p "TASK: 현재 폴더의 Markdown 파일 수만 보고한다. CONTEXT: 현재 폴더만 읽는다. SCOPE: 수정 금지. VERIFY: 없음. REPORT: 마지막 줄 DONE: <개수>."
```

수정이 필요한 GLM 작업은 래퍼를 통해 권한을 명시합니다.

```
glm -p --permission-mode acceptEdits "TASK: README의 깨진 링크를 고친다. CONTEXT: README.md만 읽고 수정한다. SCOPE: 확인 가능한 링크 표기만 고친다. OUT-OF-SCOPE: 내용 재작성 금지. VERIFY: rg -n 'old-link' README.md. REPORT: 마지막 줄 DONE 또는 FAILED."
```

[실패 신호] 워커가 계획만 말한 뒤 종료하거나, 권한 요청에서 멈춥니다. 또는 사람이 대화형 창을 계속 지켜봐야 다음 단계가 진행됩니다.

[복구] -p와 --permission-mode acceptEdits의 위치를 다시 확인합니다. prompt에 수정 허용 파일과 “승인 대기 금지”를 넣습니다. 그래도 권한 경계가 필요한 위험 작업은 자동화하지 말고 사람에게 넘깁니다.

GLM 래퍼는 endpoint와 문맥 크기를 고정합니다

[왜] GLM을 Claude Code 인터페이스로 호출할 때 endpoint, 인증 전달, 모델 ID, 작은 모델 기본값, 문맥 한도를 매번 손으로 맞추면 실수가 생깁니다. 특히 종량제 endpoint를 잘못 쓰거나 작은 보조 모델로 조용히 바뀌는 문제를 막아야 합니다.

[규칙] 이 장은 build/glm_wrapper.txt의 동작을 기준으로 합니다. GLM은 glm 래퍼로만 호출합니다. 래퍼는 구독 경로 <https://api.z.ai/api/anthropic>와 관련 환경변수를 프로세스 범위에 설정하고, 기본 모델과 작은 보조 모델을 모두 glm-5.3으로 고정합니다. 인증값은 코드에 쓰지 않고 부모 셸의 ZAI_API_KEY에서만 받습니다.

[명령/코드] 모델을 명시해야 하는 경우에도 키는 셸 환경에만 둡니다.

```
GLM_MODEL=glm-5.3 glm -p --permission-mode acceptEdits "TASK: 테스트 오류 메시지를 정리한다. CONTEXT: tests/errors.test.ts만 수정한다. SCOPE: 예상 메시지와 assertion만 맞춘다. OUT-OF-SCOPE: 애플리케이션 코드 수정 금지. VERIFY: npm test -- errors. REPORT: 마지막 줄 DONE 또는 FAILED."
```

래퍼 전문은 구매자 파일 templates/glm_wrapper.sh(12장에도 수록)에 있습니다. ZAI_API_KEY는 셸 시작 파일의 환경변수로만 두고 스크립트 안에 값을 적지 않습니다.

[실패 신호] 로그에 `[claude-code:unrecognized_model]`가 보였다는 이유만으로 실패로 판정하거나, `settings.json`의 `modelOverrides`에 GLM 이름을 추가해 해결하려 합니다.

[복구] 이 경고는 Claude Code가 Anthropic 모델 ID만 인식하면서 나타날 수 있는 정보성 메시지입니다. 응답 본문과 `exit=0`을 먼저 확인합니다. 실제 실패는 무응답 또는 `exit≠0`입니다. 종량제 `/api/paas/v4`로 우회하거나 모델 매핑을 임의로 추가하지 않습니다.

GLM smoke test

[왜] 래퍼는 환경변수와 외부 endpoint를 함께 다루므로, 명령 존재 확인만으로는 인증·모델·응답 경로가 맞는지 알 수 없습니다.

[규칙] smoke test는 한 문장 답을 요청하고, 출력에 비밀정보를 포함하지 않습니다. 경고가 있어도 exit code와 결론 본문으로 판정합니다.

[명령/코드] 다음은 파일을 바꾸지 않는 점검입니다.

```
glm -p "TASK: '워커 준비됨'이라는 문장을 출력한다. CONTEXT: 파일을 읽지 않는다. SCOPE: 파일 수정과 외부 호출 금지. VERIFY: 없음. REPORT: 마지막 줄 DONE: smoke test."
```

[실패 신호] 응답이 없거나 프로세스가 0이 아닌 코드로 끝납니다. 키가 없다는 메시지가 나오면 환경에 키가 없을 뿐, 모델 경고와 같은 종류의 오류가 아닙니다.

[복구] `ZAI_API_KEY=SET` 여부만 확인하고 개인 셸 시작 파일의 설정을 점검합니다. 값을 복사해 채팅이나 로그에 붙이지 않습니다. endpoint, 키, 모델, 문맥 한도 중 어느 단계가 실패했는지 분리해 기록합니다.

5. agy와 Gemini 계열 보조 워커

agy는 **Google Gemini 계열 CLI**(Antigravity CLI, 명령 `agy`)입니다. 이 책에서 agy는 조사·코드 탐색·문서화를 맡는 보조 워커를 뜻합니다. 표준 Gemini CLI의 명령 `gemini`와 이름이 다르므로 같은 실행 파일로 오해하면 안 됩니다.

agy는 조사와 문서화에 배정합니다

[왜] 코드베이스 탐색과 자료 정리는 구현 워커의 문맥을 불필요하게 채웁니다. agy를 별도 읽기·문서 워커로 두면 Sol과 Terra는 수정과 테스트에 집중할 수 있습니다.

[규칙] agy에는 읽을 디렉터리, 산출 형식, 수정 금지를 명시합니다. 제공된 Claude 계열 도움말에서 확인되는 `--dangerously-skip-permissions`와 헤드리스 `-p`만 사용합니다. 권한 경계를 넓히는 호출은 신뢰하는 프로젝트에서만 쓰고, 계정·브라우저·비밀 파일은 범위에서 제외합니다.

[명령/코드] 영향 분석을 맡기는 예시입니다.

```
agy --dangerously-skip-permissions -p "TASK: 알림 API 변경의 호출 지점을 조사한다. CONTEXT: src/notifications와 docs/api.md만 읽는다. SCOPE: 파일별 영향과 회귀 위험을 Markdown 표로 보고한다. OUT-OF-SCOPE: 파일 수정, 비밀 파일 읽기, 브라우저 사용 금지. VERIFY: 없음. REPORT: 마지막 줄 DONE: <요약>."
```

[실패 신호] agy가 수정까지 시작하거나, 조사 범위를 정하지 않아 대형 저장소 전체를 순회합니다.

[복구] OUT-OF-SCOPE: 파일 수정 금지를 분명히 하고 읽을 경로를 두세 개로 좁힙니다. 결과가 너무 넓으면 다음 호출에 새 도구를 추가하지 말고 질문 하나만 줄여 보냅니다.

Gemini CLI/agy도 실제 출력으로 확인합니다

[왜] 설치 안내는 오래될 수 있고, 로그인 상태는 터미널마다 다를 수 있습니다. 이름이 실행된다는 사실은 prompt가 처리되었다는 증거가 아닙니다.

[규칙] Gemini 계열 보조 워커는 제공된 agy 인터페이스로 짧은 읽기 전용 prompt를 실행해 확인합니다. 모델을 추측해 별도 플래그로 지정하지 않습니다. 현재 도움말과 조직의 래퍼가 지원하는 호출만 사용합니다.

[명령/코드] 다음 smoke test는 안전한 결론 본문을 요구합니다.

```
agy --dangerously-skip-permissions -p "TASK: '보조 워커 준비됨'이라고 한 줄로 답한다. CONTEXT: 파일을 읽지 않는
다. SCOPE: 수정 · 브라우저 · 계정 작업 금지. VERIFY: 없음. REPORT: 마지막 줄 DONE: smoke test."
```

[실패 신호] 프로세스는 끝났지만 결론 본문이 없거나, 인증·권한 오류가 나타납니다.

[복구] 종료 코드와 마지막 본문을 확인합니다. 인증은 사람이 제공자의 정상 로그인 절차로 해결합니다. 지휘관은 자격 증명을 대신 생성하거나 회피하지 않습니다.

6. 셸 시작 파일에 안전하게 연결하기

환경변수는 개인 범위에 둡니다

[왜] 여러 터미널에서 워커를 쓰려면 인증값이 새 셸에도 있어야 합니다. 그러나 프로젝트의 .env나 README에 넣으면 Git, 백업, 협업자, 워커 문맥으로 퍼질 수 있습니다.

[규칙] 개인 셸 시작 파일에는 비밀값 자체가 아니라 승인된 비밀 저장소를 읽는 초기화만 둡니다. 원문 키를 ~/.zshrc, 프로젝트 .env, 문서에 직접 쓰지 않습니다. 새 셸에서 주입한 뒤 값 대신 존재 상태만 검사합니다.

[명령/코드] 별도 비밀 저장소가 준비되지 않은 smoke test에서는 zsh가 입력을 화면에 표시하지 않게 받아 현재 셸에만 둡니다. 이 값은 셸이 끝나면 사라집니다.

```
read -r -s "ZAI_API_KEY?ZAI API key: "
printf '\n'
export ZAI_API_KEY
```

적용 뒤에는 존재만 확인합니다.

```
if [ -n "$ZAI_API_KEY" ]; then echo "ZAI_API_KEY=SET"; else echo "ZAI_API_KEY=missing"; fi
```

[실패 신호] 설정 파일을 고쳤는데 새 터미널에서 missing이 나오거나, 키를 확인하려고 실제 값을 출력합니다.

[복구] 사용 중인 셸이 zsh인지 확인하고, 새 터미널에서 다시 검사합니다. 값 자체는 출력하지 않습니다. 프로젝트별 환경 파일과 개인 시작 파일이 충돌하면 어떤 쪽이 로드되는지 존재 상태와 파일 권한만 확인합니다.

smoke test는 한 번에 한 워커씩 합니다

[왜] 처음부터 다섯 워커를 동시에 실행하면 어느 인증, 어느 래퍼, 어느 권한 경계가 실패했는지 분리할 수 없습니다. 설치 단계의 병렬성은 빠른 것보다 진단 가능한 것이 중요합니다.

[규칙] codex Luna, Claude Code, GLM, agy 순서처럼 하나씩 smoke test를 실행합니다. 각 결과의 종료 코드와 결론 본문을 읽은 뒤 다음 워커로 넘어갑니다. 설치 확인 후 실제 작업에서는 독립 작업을 병렬화합니다.

[명령/코드] 점검 결과를 다음 표로 기록합니다.

워커	모델 또는 인터페이스	smoke test	exit code	결론 본문	판정
codex	gpt-5.6-luna	읽기 전용 prompt	기록	읽음	준비/차단
Claude Code	<code>claude -p</code>	한 줄 응답	기록	읽음	준비/차단
GLM	glm-5.3	한 줄 응답	기록	읽음	준비/차단
agy	<code>agy --dangerously-skip-permissions -p "..."</code>	한 줄 응답	기록	읽음	준비/차단

[실패 신호] 표에 “실행함”만 있고 종료 코드와 결과 본문이 없습니다.

[복구] 준비 판정을 취소하고 해당 워커의 로그 마지막 부분을 읽습니다. 원인이 키, 모델, cwd, 권한, 네트워크 중 무엇인지 하나로 좁힌 뒤에만 다시 실행합니다.

7. 첫 병렬 실행을 위한 운영 규칙

먼저 파일 소유권을 나눕니다

[왜] 워커가 준비되었다고 곧바로 같은 저장소에 다섯 prompt를 보내면, 충돌은 설치 문제가 아니라 운영 설계 문제가 됩니다. 마지막으로 저장한 워커의 변경이 다른 결과를 덮을 수 있습니다.

[규칙] 첫 병렬 실행은 서로 다른 파일을 수정하는 세 작업으로 시작합니다. 추가 워커에는 읽기 전용 조사나 적대적 리뷰를 배정합니다. 중요한 동일 작업의 이중 구현은 별도 worktree에서만 합니다.

[명령/코드] 파일 소유권을 먼저 적습니다.

```
Sol: src/api/error.ts
Terra: tests/api/error.test.ts
Luna: docs/api.md
GLM: 읽기 전용 영향 분석
agy: 읽기 전용 회귀 위험 조사
```

[실패 신호] 한 워커가 테스트 파일을 고치는 동안 다른 워커도 같은 assertion을 바꾸고 있습니다.

[복구] 한 작업을 중지하고 소유권을 다시 배정합니다. 이미 충돌한 변경은 두 결과를 비교한 뒤 한 쪽을 기준으로 검증합니다. 자동 병합을 신뢰하지 않습니다.

세팅 단계에서는 종료 코드와 결론만 기록합니다

[왜] 세팅 장에서 완료 판정까지 반복하면 5장의 감시 규칙과 충돌하기 쉽습니다.

[규칙] smoke test 표에는 프로세스 종료 코드와 결론 본문만 기록합니다. codex의 `tokens used`와 비-codex의 결론 본문을 나누는 전체 상태 전이는 → 5장을 참고합니다.

[명령/코드] 파이프라인으로 출력을 줄여야 한다면 원래 명령의 실패가 가려지지 않게 실행합니다.

```
(
  set -o pipefail
  npm test 2>&1 | tail -n 5
  verify_status=$?
  printf 'exit=%s\n' "$verify_status"
  exit "$verify_status"
)
```

[실패 신호] `tail`의 성공을 smoke test 성공으로 기록하거나, 종료한 워커의 결론을 읽지 않습니다.

[복구] 해당 행의 준비 판정을 취소하고 원래 명령의 종료 코드와 결론을 다시 수집합니다. 워커별 마커 판정은 5장의 단발 감시 스크립트로 넘깁니다.

8. 세팅 완료의 기준

세팅 완료는 “도구를 설치했다”가 아닙니다. 안전한 디렉터리에서 각 워커가 비밀값을 드러내지 않고 prompt를 처리하며, 지휘관이 결과를 판정할 수 있다는 상태입니다. 모델의 속도나 응답 길이는 smoke test의 수용 기준이 아닙니다.

마지막으로 실행하는 실제 업무도 작은 것부터 시작합니다. 문서 한 파일, 단위 테스트 한 파일, 읽기 전용 영향 조사처럼 되돌리기 쉬운 작업을 병렬로 보내고, 지휘관이 diff와 종료 코드를 확인합니다. 그 한 번이 통과한 뒤에만 더 큰 기능을 워커 풀에 나눕니다.

체크리스트

- 모든 워커를 `cd`로 확인한 프로젝트 루트에서 호출했는가?
- codex·Claude Code·Gemini CLI를 실제 설치하고 선택한 구독·인증 경로를 확인했는가?
- codex 호출에 모델 ID를 명시하고 중요한 Sol 작업에는 effort를 명시했는가?
- Claude Code 헤드리스 작업에 `-p`, 수정 작업에 `--permission-mode acceptEdits`를 썼는가?
- GLM은 `glm-5.3`과 구독용 래퍼로만 호출했는가?
- API 키·토큰을 출력하거나 prompt·저장소·문서에 넣지 않았는가?
- 각 smoke test에서 exit code와 결론 본문을 모두 읽었는가?
- 실제 병렬 실행 전에 파일 소유권과 검증 명령을 나눴는가?

흔한 실패 3가지

1. 기본 설정을 믿고 모델과 effort를 생략합니다. 중요 호출에는 모델 ID와 필요한 effort를 명시해 재현성을 확보합니다.

>

2. GLM 경고를 실패로 오해합니다. unrecognized_model은 정보성 경고일 수 있습니다. 응답 본문과 exit=0을 함께 확인합니다.

>

3. 키를 문제 해결용 로그에 출력합니다. 값은 절대 보지 말고 SET 또는 missing만 확인합니다. 노출 의심 시에는 즉시 교체합니다.

미리보기는 여기까지입니다

전체 13장과 템플릿 7종은 판매 페이지에서 구매하실 수 있습니다. 구매 링크는 GitHub 저장소 README에 있습니다.